

Lecture 5: an outline

- ▶ Sparse matrices.
 - ▶ Occurrence, need of them.
 - ▶ Operations with these.
- ▶ Linear systems.
 - ▶ Matrix representation.
 - ▶ Numerical solution, number of operations.

Lecture 5: Sparse matrices: why?

Observation: Many times, the vast majority of elements in a matrix are zero.

- ▶ The matrix A in the analysis of purchases:
 - ▶ Rows correspond to customers.
 - ▶ People with registration or with a .. card can be identified.
 - ▶ Columns to a product (in a supermarket or a webshop).
 - ▶ $A_{j,k} = P$ if customer $\#j$ has bought a number P product $\#k$.
 - ▶ A bill that you (as customer $\#j$) get:
 - ▶ the sparse representation of $A[j, :]$.

Lecture 5: Sparse matrices: another example

- ▶ Network analysis (graph representation):
 - ▶ Rows and columns correspond to nodes/people/hubs.
 - ▶ $A_{j,k} = 1$ if node $\#j$ and $\#k$ are connected,
 - ▶ and zero otherwise.
 - ▶ Number of non-zeros = number of connections (“edges”)
 - ▶ A should be symmetric.
 - ▶ Upper side would be enough to store.

Lecture 5: Sparse matrices: how

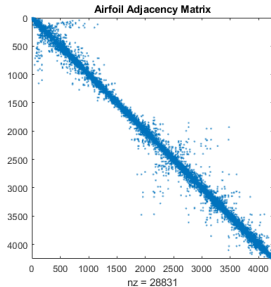
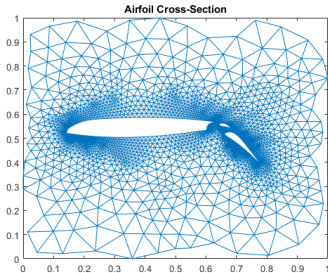
- ▶ Efficient computing: we need to store only nonzero elements (their indices).
- ▶ Commands in Python: `csr_matrix`, `csc_matrix`
 - ▶ Alternative formats (for different purposes): `lil_matrix`,
- ▶ Command for computing density: `count_nonzero(A) / A.size`
- ▶ One can multiply, concatenate sparse matrices
 - ▶ Without this, one may encounter memory overflow.
- ▶ Be careful, which operations preserve sparse structure.

Lecture 5: Sparse matrices: a further case

- ▶ In scientific computing (discretization for numerical solution of PDE's) we need sparse matrices.
 - ▶ They have
 - ▶ non-zero (main)diagonal elements,
 - ▶ non-zero sub-diagonal and super-diagonal elements,
 - ▶ a few more diagonals with non-zeros.
 - ▶ Just remember HW 1/4.
 - ▶ These connected with the network example: we have non-zero entries for the indices of neighboring elements in the computational grid.
 - ▶ There are separate command to create such matrices by hand.
 - ▶ One can define diagonals directly.
 - ▶ One can use tricks (e.g. Kronecker product).
 - ▶ Construction is non-trivial: computing elements and collecting such a matrix (e.g. of size $10^{12} \times 10^{12}$) takes much time.

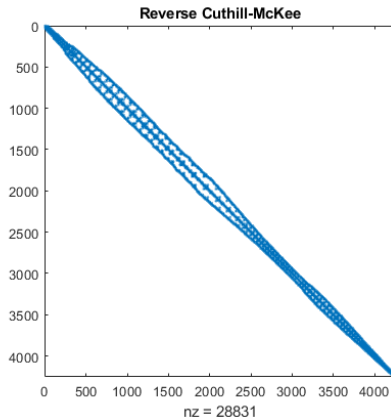
Lecture 5: Sparse matrices: a real-life case

- ▶ A real-life example: the space around an airfoil is discretized into a triangle (tetrahedral) grid with gridpoints $\{\mathbf{x}_k\}$.
- ▶ The connectivity matrix of this: $A[j, k] \neq 0 \Leftrightarrow \mathbf{x}_j$ and $\mathbf{x}_k$ share an edge. Non-zero locations are colored.



- ▶ Sometimes, this is still not nice: one should permute the numbering such that points with close indices are connected.

Lecture 5: an example for a “better” numbering



- This matrix is easier to deal with (e.g. for solving a linear system)

Lecture 5: Sparse matrices - an important takeaway

Efficient computations in Python:

- ▶ Use vectorized operations.
- ▶ Use sparsity of matrices if they arise.

Lecture 5: Linear systems - motivation

- ▶ An example: we want to know the **value** of some **factors** in prizes of office buildings.
- ▶ **Factors**: total groundspace, number of elevators, number of parking places, income from existing rent contracts, number of restaurants around, distance from public transport, maintaining costs.
- ▶ Summarized: We have 7 **factors**: $f_1, f_2, \dots, f_7$
- ▶ We try to find the **values** $w_1, w_2, \dots, w_7$ of them.
- ▶ We assume that price depends linearly from the quantity of these factors meaning that
$$\text{price} = f_1 w_1 + f_2 w_2 + \dots + f_7 w_7 = \mathbf{f} \cdot \mathbf{w}.$$
- ▶ We have a data set of contracts:
 - ▶ Correspond to given office buildings (#30).
 - ▶ Given **factors**, given prices $(p_1, p_2, \dots, p_N) := \mathbf{p}$.
 - ▶ Unknown values.

Lecture 5: Linear systems - an example

- ▶ Define $M \in \mathbb{R}^{30 \times 7}$:
 - ▶ $M_{j,k}$ = k th factor of the j th office building.
 - ▶ Rows correspond to contracts, columns to the factors.
- ▶ Price of j th office building: $p_j = M[j, :] \cdot \mathbf{w}^T$.
- ▶ In matrix-vector form: $M \cdot \mathbf{w}^T = \mathbf{p}$.
- ▶ Linear systems of equations: matrix-vector equations
 - ▶ with known matrix M and right-hand side $\mathbf{p}$,
 - ▶ with unknown vector $\mathbf{w}^T$ on the left hand side.

Lecture 5: Linear systems - example continued

- ▶ In real-life situations:
 - ▶ # of rows $\gg$ # of columns.
 - ▶ We have a large data set, and want to compress the information from this.
 - ▶ Indeed, we cannot expect to have a solution.
 - ▶ It is called: over-determined system.
- ▶ In this case, we should not use $=$ sign, and ask instead:
Find $\mathbf{w}^T$ such that the approximation
 $\mathbf{M} \cdot \mathbf{w}^T \approx \mathbf{p}$ is the best possible.
 - ▶ How to understand this?
 - ▶ Indeed, the difference $\mathbf{M} \cdot \mathbf{w}^T - \mathbf{p}$ should be minimal in some sense.
 - ▶ We will discuss it later.

Lecture 5: Linear systems - if they can be solved I.

- ▶ An ideal case:

of unknowns = # of equations

- ▶ We have a square matrix.

- ▶ This is invertible.

- Then there is a unique solution.

- ▶ In scientific computing, this is frequently the case (remember to the airfoil, grid, matrix, ...).

- ▶ OK, then solve $A\mathbf{x} = \mathbf{b}$.

- ▶ Formally take simply $\mathbf{x} = A^{-1}\mathbf{b}$.

- ▶ A^{-1} should also be computed.

- ▶ Never do that: this is expensive to compute.

- ▶ There is a command (in Python) for solving linear systems without taking inverse.

Lecture 5: Linear systems - if they can be solved I.

- ▶ Simplest case:
 - ▶ eliminate variables after each other:
 - ▶ express x_1 with the remaining ones using eq. 1
 - ▶ put into eq. 2 and express x_2 with the remaining ones
 - ▶ finally we get the last one.
 - ▶ Substitute this back getting the previous one.
 - ▶ Repeat it to have all of the unknowns.
 - ▶ Called direct methods.
 - ▶ Tricky versions are still used to solve very large and very sparse systems.
- ▶ Real-life cases: Iterative *approximations* of the solution.

Lecture 5: Linear systems: iterative approximations - basics

Basic idea:

- ▶ Given a linear system $A\mathbf{x} = \mathbf{b}$.
- ▶ We try to find another matrix B and a vector $\mathbf{c}$ such that
 - ▶ B is invertible.
 - ▶ The unique solution of $A\mathbf{x} = \mathbf{b}$ is identical to the unique solution $\mathbf{u}$ of $\mathbf{u} = \textcolor{red}{B}\mathbf{u} + \textcolor{blue}{c}$.
 - ▶ We do not solve it directly either, but instead:
 - ▶ An iteration $\mathbf{u}^{n+1} = B\mathbf{u}^n + \mathbf{c}$ is applied.
 - ▶ We will give sufficient condition to ensure its convergence.

Lecture 5: Linear systems: iterative approximations - an example

Basic idea:

- ▶ Given a linear system $A\mathbf{x} = \mathbf{b}$,
- ▶ where for each diagonal element we have
$$|A_{j,j}| > |A_{j,1}| + |A_{j,j-1}| + \cdots + |A_{j,N}|.$$
 - ▶ This is a strict assumption (but true for HW 1.(d)).
- ▶ We make this choice:
 - ▶ D denotes the diagonal of A (as a matrix).
 - ▶ $\mathbf{c} = D^{-1}\mathbf{b}$,
 - ▶ $\mathbf{B} = -D^{-1}(A - D)$.
- ▶ If $A\mathbf{x} = \mathbf{b}$, then $\mathbf{0} = -D^{-1}A\mathbf{x} + D^{-1}\mathbf{b}$, which gives

$$\mathbf{x} = -D^{-1}A\mathbf{x} + D^{-1}D\mathbf{x} + D^{-1}\mathbf{b} = -D^{-1}(A - D)\mathbf{x} + D^{-1}\mathbf{b}.$$

Lecture 5: Linear systems: iterative approximations - an example (continued)

Theorem

Using the above conditions the iteration

$$\mathbf{x}^{n+1} = -D^{-1}(A - D)\mathbf{x}^n + D^{-1}\mathbf{b}$$

has a unique fixed point, which satisfies

$\mathbf{x} = -D^{-1}(A - D)\mathbf{x} + D^{-1}\mathbf{b}$ and therefore, solves $A\mathbf{x} = \mathbf{b}$.

► The above iterative method: Jacobi iteration.

► Computational error:

$$\frac{\|\mathbf{x} - \mathbf{x}^{n+1}\|}{\|\mathbf{x} - \mathbf{x}^n\|} \leq \rho < 1, \text{ with } \rho \text{ depending on } -D^{-1}(A - D).$$

► A number of further (faster) iterative methods exist.

Lecture 6: an outline

- ▶ Eigenvalue problems.
 - ▶ Real and complex eigenvalues.
- ▶ Eigenvalues of stochastic matrices.
- ▶ Numerical approximation of eigenvalues.

Lecture 6: eigenvalues

- ▶ Back to the geometrical interpretation:
 - ▶ $A \in \mathbb{R}^{n \times n}$, $A\mathbf{x} \in \mathbb{R}^n$.
- ▶ It is possible that $A\mathbf{x} = \lambda\mathbf{x}$ for some number λ and $\mathbf{x} \neq \mathbf{0}$.
 - ▶ Such a λ is called an *eigenvalue* of A ,
 - ▶ a corresponding $\mathbf{x} \neq \mathbf{0}$ is called an *eigenvector*.
- ▶ Of course: if $\mathbf{x}$ is an *eigenvector*, $3\mathbf{x}$ is also an *eigenvector*.
 - ▶ Satisfactory: eigenvectors of length one.

What is wrong here? If 0 is an eigenvalue and $\mathbf{x} \neq \mathbf{0}$ an eigenvector:

$$\mathbf{x} = A^{-1}A\mathbf{x} = A^{-1} \mathbf{0} = \mathbf{0}$$

- ▶ This is a contradiction.
 - ▶ Indeed, in this case, A^{-1} cannot exist.

Lecture 6: eigenvalues - characterization

- ▶ If $A\mathbf{x} = \lambda\mathbf{x}$
 - ▶ $\mathbf{0} = A\mathbf{x} - \lambda\mathbf{x} = (A - \lambda I)\mathbf{x}$.
 - ▶ We try to find λ such that $\text{rank}(A - \lambda I) < n$, such that we have a solution $\mathbf{x} \neq \mathbf{0}$.
- ▶ It cannot have a unique solution:
 - ▶ $\mathbf{x}$ is not unique as mentioned.
 - ▶ Moreover, for an eigenvalue, more linearly independent eigenvectors may exist (think, e.g., of I).
Then the eigenvalue is called *multiple eigenvalue* (*geometrically*).
- ▶ This can be solved, but in practice, would be too long.
- ▶ Notation: sort the eigenvalues of A (if λ_1 is not a multiple one):
$$|\lambda_1| > |\lambda_2| \geq |\lambda_3| \geq \dots$$

Lecture 6: eigenvalues - example, special cases

- ▶ An easy example: $Q = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$
 - ▶ $\mathbf{0} = (Q - \lambda I)\mathbf{x} = \begin{pmatrix} -\lambda & -1 \\ 1 & -\lambda \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} -\lambda x_1 - x_2 \\ x_1 - \lambda x_2 \end{pmatrix}.$
 - ▶ Multiplying the first one with $-\lambda$ and adding to the second:
 $(1 + \lambda^2)x_1 = 0 \Rightarrow \lambda^2 = -1.$
 - ▶ Eigenvalues of Q : $\lambda_1 = i, \lambda_2 = -i.$
- ▶ One could expect it: Q corresponds to a rotation; (in non-trivial cases) it never only multiplies vectors.
- ▶ If $A \in \mathbb{R}^{n \times n}$ is symmetric: all of its eigenvalues are real.

Lecture 6: eigenvalues - special cases (continued)

- ▶ Any right stochastic matrix $S \in \mathbb{R}^{n \times n}$ has the eigenvector $\mathbf{1}$ with the corresponding eigenvalue 1.
- ▶ Indeed: taking $S \cdot \mathbf{1}$, its j th element is $S[j, :] \cdot \mathbf{1}$, the sum of the element in the j th row of S .
- ▶ This is *one* for S ; therefore we get back $\mathbf{1}$.
- ▶ Also, we have that for any eigenvalue λ : $|\lambda| \leq 1$.
- ▶ This should be valid also for doubly stochastic matrices.
- ▶ Left stochastic matrices have also the eigenvalue 1.
 - ▶ Finding a corresponding eigenvector is not trivial, but important.

Lecture 6: Find eigenvalues - a simple numerical method

- ▶ Task: Find the eigenvalue λ_1 with the largest absolute value and the corresponding eigenvector $\mathbf{x}$.
- ▶ Condition for working this algorithm: $\mathbf{x}$ should be unique up to a constant factor.
- ▶ The algorithm (*power method*):
 - ▶ Take an arbitrary unit vector $\mathbf{v}_0$.
 - ▶ Compute $\mathbf{v}_{n+1} := \frac{A\mathbf{v}_n}{|A\mathbf{v}_n|}$.
 - ▶ Again a unit vector.
 - ▶ Quick operation (esp. for a sparse matrix A).
- ▶ Convergence rate: $\left(\frac{|\lambda_2|}{|\lambda_1|}\right)^n$.
 - ▶ Meaning of this:
$$|\mathbf{x} - \mathbf{v}_n| \sim \left(\frac{|\lambda_2|}{|\lambda_1|}\right)^n$$
 - ▶ This is exponential approximation rate.